

A new specification for `std::generate_canonical`

This paper proposes a new specification for the function template `generate_canonical` [rand.util.canonical, 28.5.8.2]. The outcome is preserved (namely a random number in the interval $[0, 1)$), but the algorithm and the constraints are changed to ensure the desired statistical properties. The new specification obsoletes [LWG 2524](#).

Contents

1. [Problem statement](#)
2. [Drive-by fixes](#)
3. [DO NOT SUBMIT: Proposed naive wording](#)
4. [Statistical flaws in the current specification](#)
5. [A new specification](#)
6. [Impact on the Standard](#)
7. [Questions for LWG and SG6](#)
8. [Proposed wording \(not included\)](#)

Problem statement

The specification of `generate_canonical` in C++23 and in the current working paper (N4958) is effectively unimplementable since it is over-constrained. To be more precise, it is *wrongly* constrained in terms of purely mathematical expressions that ignore the reality of floating point rounding on real implementations. We will point out two problems with the current specification. The first one is immediate, the second is somewhat more subtle and will be discussed later in this paper. First, consider the the following three current requirements.

1. The result must lie in $[0, 1)$.

2. The algorithm is specified exactly and the underlying URBG must be invoked a specific, fixed number of times for a given set of parameters.
3. The results must be uniformly distributed.

The immediate problem is that this is unimplementable for the following reasons: The algorithm is currently specified exactly as a particular computation, which results in a fraction S/R^k that is mathematically guaranteed to be less than 1. However, the value may be arbitrarily close to 1, and when expressed as a value of a bounded-precision type in C++, the result may actually be exactly 1 due to rounding. (This causes real bugs, e.g. when a computation divides by $(1 - x)$, where x was obtained from `generate_canonical`.) If we accept (2), the algorithm as written, the rounding violates constraint (1). If we modify the result when the algorithm results in 1, we violate (3), uniformity. If we want to preserve (1) and (3), we need to rerun the algorithm in the case where it results in 1, which violates (2), the precise computational prescription.

LWG decided at the 2017 Albuquerque meeting that the best solution is to change the specification to rerun the algorithm until the result is not equal to 1. This means that the complexity of the algorithm can no longer be stated precisely, but only in expectation. If one round of the algorithm invokes the URBG k times, and the result is 1 with probability $(1 - p)$, then the expected number of invocations of the URBG is now k/p . (In practice, p will be very close to (and less than) 1.) This paper attempts to improve on that decision by also addressing some statistical issues.

Drive-by fixes

The original expression for b assumes that the radix is 2 when comparing `numeric_limits<RealType>::digits` to the template parameter `bits`. We propose to interpret that template parameter as a number of digits in the radix `numeric_limits<RealType>::radix`; SG6 concurred with this direction in March 2022. Accordingly, we rename it to `digits`, and we also rename the variable used in the specification from b to d .

DO NOT SUBMIT: Proposed naive wording

If we simply modified the specification to rerun the algorithm until it results in a value less than 1, we could use the following wording. However, this wording fails to address the radix issue or the deeper statistical problems that we will discuss below, so we do not want to keep the change as small as this. This

section is included solely for historical interest because this wording has been discussed on the reflector before.

Strike out bad wording: 

Modify [rand.util.canonical, 29.6.7.2] paragraphs 3 and 4 as follows.

```
template<class RealType, size_t bits, class URBG>  
RealType generate_canonical(URBG& g);
```

~~3. Complexity: For each attempt (see below), exactly~~**Exactly** $k = \max(1, \lceil b / \log_2 R \rceil)$ invocations of `g`, where b is the lesser of ~~`numeric_limits<RealType>::digits` and `bits` and `log2(numeric_limits<RealType>::radix) * numeric_limits<RealType>::digits`~~, and R is the value of ~~`g.max()` — `g.min()` + 1~~.

~~4. Effects: For each attempt, invokes~~**Invokes** `g()` k times to obtain values $g_0, \dots, g_{k-1}$, respectively ~~and calculates~~. **Calculates** a quantity $\{S = \dots\}$ using arithmetic of type `RealType`. ~~Attempts are repeated as long as the quantity S/R^k has the value 1.0 when expressed as type `RealType`.~~

~~5. Returns: S/R^k .~~

~~6. Throws: What and when `g` throws.~~

Statistical flaws in the current specification

The currently specified algorithm does not always result in uniform output due to rounding. To see this, consider first a simple lemma.

Claim. *The restriction of a uniform distribution on a finite set to a subset is uniform.*■

We use this to look for statistical properties of `generate_canonical`. If the function generates uniformly distributed floats in the range $[0, 1)$, then by throwing away all numbers less than 0.5, we retain a uniform distribution on the set $[0.5, 1)$. In the popular IEEE-754 floating-point representation, numbers in this range have a fixed exponent (of effective value -1), and so we get a uniform distribution of mantissas, and thus each bit of the mantissa (or perhaps of a leading subset of significant bits, when the algorithm is used with low precision) is independently uniformly distributed. This is a property we can look for. More

generally, similar statements should hold for restrictions to intervals of the form $[2^{-n}, 2^{-n} + 1)$.

The problem in the current specification comes from the use of division combined with floating point rounding. Whenever a larger range is used to derive a smaller range via division (rather than just discarding bits, when that is an option), the floating point rounding behaviour affects the results. To illustrate, consider the sequence 0.0, 0.5, 1.0, 1.5, 2.0, 2.5, 3.0, 3.5. If we *discard* the least significant bit, we obtain 0, 0, 1, 1, 2, 2, 3, 3, which is uniform. But if we employ the popular to-nearest-even rounding, we obtain 0, 0, 1, 2, 2, 2, 3, 4, which biases even to odd numbers at a ratio of 3-to-1. (This also shows another bias, namely that 0 only gets hit twice.)

To make this problem concrete, consider a typical 32-bit float with 24 mantissa digits. For an extreme case, if the URBG returns 25 bits, then in the restricted range $[0.5, 1.0)$ the least significant bit results from rounding away the last digit of g_0 in just the same way as in the previous example, which leads to a 3-to-1 bias of zeros over ones in the last bit. A 26-bit URBG would have to strip two bits in the range $[0.5, 1.0)$ (leading to a 5-to-3 bias), but only one bit in the range $[0.25, 0.5)$, and so on. With the typical 32-bit URBG, the bias in the last bit shows up when restricting to the range $[2^{-8}, 2^{-7})$ (or `[0x1p-8, 0x1p-7)` in code).

Other rounding modes may be fairer when rounding is needed for random number generation. However, the rounding mode is not in scope of the specification and thus not under our control. Our proposed algorithm will use discard-and-retry to create a value that can be normalized without rounding.

The problem with rounding to nearest-even was also discovered in [the Java library's nextDouble function](#), which originally used a non-trivial division and would thus experience a rounding-induced bias in the lowest bit. (In the corrected, present version, the division by `1L << 53` is just a trivial exponent adjustment.)

A new specification

We propose a new specification that provably results in uniform output, does not suffer from rounding problems, and is independent of the radix of the floating point implementation. The algorithm does not use non-trivial division to produce a limited output range. Rather, it will discard any result that falls outside the desired range and retry, so that the final division does not round. This ensures uniformity and avoids any dependency on floating-point rounding behaviour. An

extra integer division (usually just a shift) is used to prevent an unreasonable expected number of retries in certain cases.

The set of possible outcomes is not required to contain *every* representable value of `RealType` in the interval $[0, 1)$. It is difficult to define uniformity over such a distribution, and providing it would be inordinately complicated in service of very unlikely, very small results. Instead, the resulting values will on common implementations be precisely the values $2^{-b}\{0, \dots, 2^b - 1\}$, where b is `digits` (restricted to the precision of the type). For example, for `digits = 2` the outcome on a typical implementation is (uniformly) 0, 0.25, 0.5 and 0.75. Note also that the mean of the resulting distribution is smaller (by 2^{-b-1}) than the ideal mean 0.5.

The proposed algorithm is as follows.

- Let r be `numeric_limits<RealType>::radix`, let d be the smaller of `digits` and `numeric_limits<RealType>::digits`, and let R be `g.max() - g.min() + 1`.
- Let k be the smallest integer such that $R^k \geq r^d$.
- Let x be $\lfloor R^k / r^d \rfloor$, which is in $(0, R)$ and need not be a power of r .

Now compute $S = \sum_{i \in [0, k)} g_i R^i$ in unbounded precision. Whenever $S \geq x r^d$, discard the result and retry; this occurs with probability less than 1/2 because $(2x) r^d > R^k$. The return value is $\lfloor S / x \rfloor / r^d$, which can be computed without rounding since $d \leq \text{numeric_limits}<\text{RealType}>::\text{digits}$.

For the edge case `digits = 0`:

- $d = 0$,
- $k = 0$, so the URBG is not called,
- $x = 1$, and
- $S \equiv 0$, so the results have zero entropy.

Note that on the most common platforms $r = 2$ and $R = 2^n$ is a power of 2, so that the definitions simplify: $k = \lceil d / n \rceil$ and $x = 2^m$, where $m = -d \bmod n$. No retries ever occur (because R^k is a multiple of r^d); the final return expression is $\lfloor S / 2^m \rfloor / 2^d$. (When d is a multiple of n , this is equivalent to the current specification.)

It is easy to see that this algorithm produces uniform outputs: The value $S < x r^d$ is obtained by restricting the URBG to the range $[0, x r^d)$. Since d does not exceed the precision of `RealType`, the final division by r^d does not round: the result is representable as a value of `RealType` strictly less than 1.

Impact on the Standard

This proposal changes the side effects, computational complexity and specific algorithmic details of the library facility `std::generate_canonical`. In particular, code that depends on a specific sequence of results from repeated invocations, or on a particular number of calls to the URBG argument, will be broken.

Questions for LWG and SG6

We requested clarification on the following details; some feedback from SG6 is presented below.

1. Is the design intention that `std::generate_canonical<RealType, bits>` picks an integer uniformly from $[0, 2^M)$ and returns the value divided by 2^M ? This is in contrast to possible alternative interpretations such as “pick a uniform real number (mathematically) from $[0, 1)$ and return its rounded `RealType` representation”. The latter suffers from the round-to-1.0 bug, of course, but it could be amended to “round-down” semantics to avoid this. However, the requirement of a mathematically uniform real number requires highly variable (and potentially large) number of URBG invocations, so we believe that this is not the design intent.

This has been the subject of some reflector discussion; SG6 confirmed this direction in March 2022.

2. How much do we want to specify the algorithm? The above description is fairly detailed and prescriptive, which we intend to result in identical results on conforming implementations. However, for certain parameters our prescription requires more retries than strictly necessary. An alternative would be to give implementations more freedom to use more efficient sampling strategies; or we could consider *mandating* more efficient sampling, at the cost of making the specification more complex.

SG6 was against removing the precise algorithm specification in March 2022.

Proposed wording

Relative to N4958.

Modify [rand.synopsis, 28.5.2] as follows.

```
// 28.5.8.2, function template generate_canonical
template<class RealType, size_t bitsdigits, class URBG>
    RealType generate_canonical(URBG& g);
```

Modify [rand.util.canonical] as follows.

```
template<class RealType, size_t bitsdigits, class URBG>
    RealType generate_canonical(URBG& g);
```

1. *Effects*: Let

- r be `numeric_limits<RealType>::radix`,
- R be `g.max() - g.min() + 1`,
- d be the smaller of `digits` and `numeric_limits<RealType>::digits`, [footnote: ~~b~~ d is introduced [...]]
- k be the smallest integer such that $R^k \geq r^d$, and
- x be $\lfloor R^k / r^d \rfloor$.

~~Invokes~~An attempt is k invocations of `g()` ~~k times~~ to obtain values $g_0, \dots, g_{k-1}$, respectively. ~~Calculates, and the calculation of~~ a quantity $[S = \dots]$ ~~using arithmetic of type `RealType`.~~ Attempts are made until $S < xr^d$.

[Note: When R is a power of r , precisely one attempt is made. —end note]

2. *Returns*: $\lfloor S/x \rfloor / R^k r^d$.

[Note: The return value c satisfies $0 \leq S/R^k c < 1$. —end note]

3. *Throws*: What and when `g` throws.

4. *Complexity*: Exactly $k = \max(1, \lceil b / \log_2 R \rceil)$ invocations of `g`, ~~where b [footnote] is the lesser of `numeric_limits<RealType>::digits` and `bits`, and R is the value of `g.max() - g.min() + 1` per attempt.~~

5. [Note: [...] —end note]

6. [Note: When R is a power of r , an implementation can avoid using an arithmetic type that is wider than the output when computing S . —end note]

Drafting note: the *Complexity* footnote is moved into the *Effects* bullet defining what is now d (the erstwhile b).